**IMT2024013**
**Brinda Perla**

# Project Report: Cinema Seat Booking Engine

## 1. Problem Statement

The objective of this project is to build a real-time "Cinema Seat Booking Engine." In a real-world scenario like a ticketing system, many users might try to book the exact same seat at the exact same time. If the system does not handle this properly, two people could end up paying for the same ticket.

This project solves that problem. It allows multiple people to run a user program and connect to one main central program at the same time. Everyone can view a live map of the cinema and book seats. To stop double-bookings, the system uses strict Operating System tools to ensure that when one person clicks on a seat, nobody else can touch it until that person finishes their booking.

## 2. Explanation of Implemented Concepts

This project fulfils all six mandatory OS concepts outlined in the project guidelines:

**2.1 Role-Based Authorisation** The system has two types of users: Admin and Customer. When a user is created, they are assigned a role integer in the user structure (0 for Admin, 1 for Customer).

- **Implementation:** After a user logs in, the server checks db_user.role. If the role is 0, the server displays the Admin Menu, which allows the admin to view the live map and unbook seats. If the role is 1, the server displays the Customer Menu, restricting them to only viewing and booking seats.

**2.2 Socket Programming** The application uses the standard C socket API to create a communication link between different terminals.

- **Implementation:** The server uses standard OS commands (socket, bind, listen, accept, connect) to create a communication link between the different programs, allowing them to send messages back and forth.

**2.3 Concurrency Control** The server must be able to talk to multiple users at the exact same time without making anyone wait in line just to view the menu.

- **Implementation:** I built a Concurrent Server using the fork() system call. Every time accept() receives a new client, fork() creates a duplicate child process. The child process handles that specific customer, while the parent process immediately goes back to waiting for new clients.

**2.4 File Locking** When a user decides they want a seat, the system must lock that specific seat record so nobody else can take it while they are making their decision.

- **Implementation:** I used fcntl() with the F_WRLCK (Write Lock) setting. When a user selects a seat, the system uses F_SETLK to try and lock just that one record in

the seats.txt file. If another user is already looking at it, fcntl() instantly returns -1, and the second user is blocked. I also used the select() system call to create a 10-second timeout. If the first user doesn't confirm the booking by typing "yes" within 10 seconds, the lock is released using F_UNLCK.

**2.5 Data Consistency** Data consistency means ensuring the binary files (users.txt and seats.txt) never get corrupted by multiple processes writing to them at the same time (Race Conditions).

- **Implementation:** I used Semaphores to protect the critical sections of the code. By calling semop() to decrease the semaphore value to -1 before a file operation, and increasing it back to 1 afterward, I ensure that only one process can successfully execute the booking code block at any given microsecond.

**2.6 Inter-Process Communication (IPC)** Because fork() creates entirely separate child processes, they cannot easily share variables. I used two IPC mechanisms to allow them to communicate.

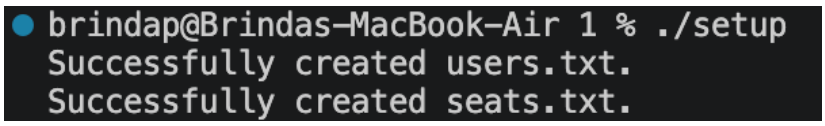
- **Implementation:**

  1. **Shared Memory (shmget / shmat):** I created a shared memory segment to hold an array of all active client Process IDs (PIDs). When a user logs in, their PID is added to the array.

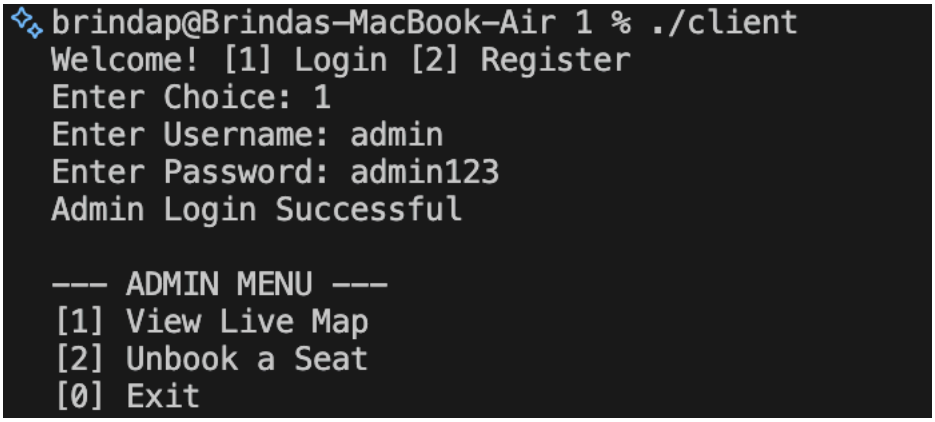
  2. **Signals (kill):** Whenever a user successfully books or unbooks a seat, the server loops through the Shared Memory array and uses the kill() system call to send a SIGUSR1 signal to all other active PIDs. This tells every other connected client to immediately refresh their screen with the updated live map.

**3. Screenshots and Output**

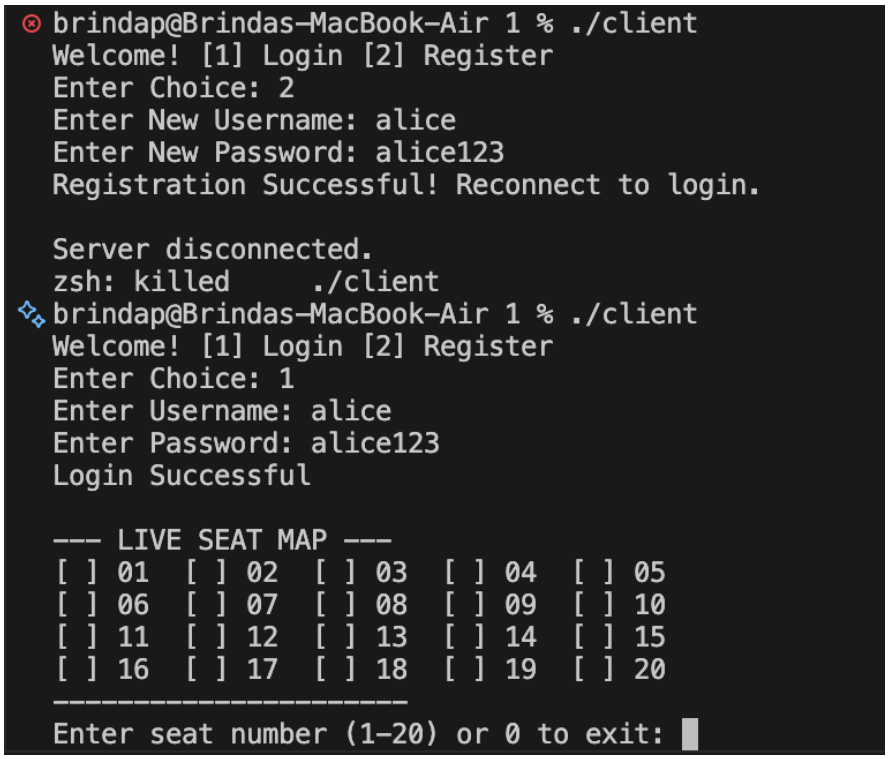
- **Screenshot 1: System Initialisation**



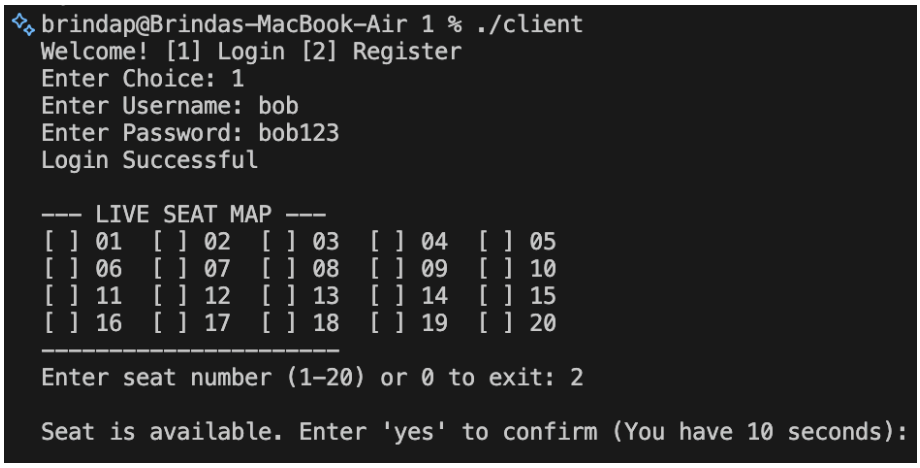

- **Screenshot 2: Role-Based Authorisation**

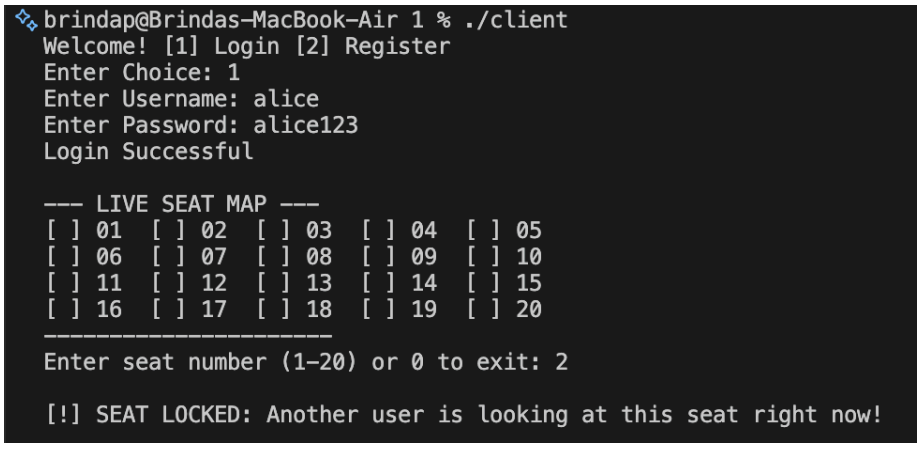


- **Screenshot 3: Live Seat Map & Booking**




- **Screenshot 4: Concurrency & File Locking in Action**



```
Seat is available. Enter 'yes' to confirm (You have 10 seconds): yes

Booking Successful

--- LIVE SEAT MAP ---
[ ] 01  [X] 02  [ ] 03  [ ] 04  [ ] 05
[ ] 06  [ ] 07  [ ] 08  [ ] 09  [ ] 10
[ ] 11  [ ] 12  [ ] 13  [ ] 14  [ ] 15
[ ] 16  [ ] 17  [ ] 18  [ ] 19  [ ] 20
----------------------
```

- **Screenshot 5: The Timeout Feature**

```
Enter seat number (1-20) or 0 to exit: 5

Seat is available. Enter 'yes' to confirm (You have 10 seconds):
Timeout! You took too long. Seat reservation cancelled.

--- LIVE SEAT MAP ---
[ ] 01  [X] 02  [ ] 03  [ ] 04  [ ] 05
[ ] 06  [ ] 07  [ ] 08  [ ] 09  [ ] 10
[ ] 11  [ ] 12  [ ] 13  [ ] 14  [ ] 15
[ ] 16  [ ] 17  [ ] 18  [ ] 19  [ ] 20
----------------------
```

- **Screenshot 6: Admin Features**

```
--- ADMIN MENU ---
[1] View Live Map
[2] Unbook a Seat
[0] Exit
Enter Choice: 1

--- LIVE SEAT MAP ---
[ ] 01  [X] 02  [ ] 03  [ ] 04  [ ] 05
[ ] 06  [ ] 07  [ ] 08  [ ] 09  [ ] 10
[ ] 11  [ ] 12  [ ] 13  [ ] 14  [ ] 15
[ ] 16  [ ] 17  [ ] 18  [ ] 19  [ ] 20
----------------------

--- ADMIN MENU ---
[1] View Live Map
[2] Unbook a Seat
[0] Exit
Enter Choice: 2
Enter seat number to unbook (1-20): 2

Seat successfully unbooked

--- ADMIN MENU ---
[1] View Live Map
[2] Unbook a Seat
[0] Exit
Enter Choice: 0

Server disconnected.
```

## 4.Challenges Faced & Solutions

### Challenge 1: Cloned Programs Cannot Share Variables

- **The Problem:** I used the fork() system call to handle multiple users at once. fork() creates a completely separate clone of the program for each user. Because they are separate, they cannot easily share normal variables. I needed a way for the main server to know exactly who was logged in.

- **The Solution:** I used Shared Memory (shmget). I created a special block of memory that acts like a shared whiteboard. Every time a new user connects, their Process ID (PID) is written on this whiteboard so all the other clones can see it.

### Challenge 2: Live Updates Crashing the 10-Second Timer

- **The Problem:** I used the select() system call to create a 10-second timer while a user decides to type "yes" to book a seat. However, if another user booked a different seat during those 10 seconds, the server would send a live update signal (kill). This incoming signal would accidentally crash the timer.

- **The Solution:** I added error checking for an "Interrupted System Call" (EINTR). I put the timer inside a loop. If a map update signal comes in and interrupts the timer, the code simply ignores the interruption and resumes the countdown safely.

### Challenge 3: Locking Bugs on Different Operating Systems

- **The Problem:** When using fcntl() to lock a seat in the file, my code was sometimes glitching. It thought a seat was locked when it was actually empty. This happened because different operating systems handle background memory differently when setting up the lock structure (struct flock).

- **The Solution:** Before setting up my lock rules, I used the memset() command to completely wipe the memory of the lock structure to zero. Then, I set my rules (like F_WRLCK and SEEK_SET) one by one. This forced the memory to be perfectly clean and fixed the bug immediately.